

Unit 2

Programming Logic

Program Development Cycle

- A correct program can be written with the proper planning and its implementation.
- Programmers develop the program in a planned manner to master the complexity of the problem.
- They follow the series of steps which is known as Program/Software Development Life Cycle(SDLC).

Steps: SDLC

1. Problem Analysis
2. Design a solution
3. Program Coding
4. Testing and Debugging
5. Implementation, maintenance and Documentation

Problem Analysis

- Also called problem definition
- Must specify the following tasks:
 - Objective
 - Output Requirement
 - Input Requirement
 - Processing Requirement
 - Evaluating Feasibility

Design a solution

- Logical design
- Programming tools: algorithm, flowchart, block diagram, Data flow diagram, E-R diagram
- Helps in coding

Program Coding

- Physical design

Testing and Debugging

- Types of error
 - Syntax error
 - Runtime error
 - Logical error
- Testing is usually performed for:
 - For verification and validation
 - To improve quality

Implementation, Maintenance and Documentation

- After debugging and testing, the overall programs are integrated.
- Finally developed program is implemented.
- Software has to go for maintenance due to change in technology, client requirement and other factors.

Documentation

- Program documentation starts from the starting of the SDLC.
- It keeps most of the information of all phases while developing projects.
- It is used for further reference for both the programmer and client.
- It should contain:
 - Problem analysis document called Software requirement Specification (SRS).
 - Program design document called Methodology
 - Program verification document using test cases
 - Log is used to document further program revision and maintenance activity.

Structure of C Program

```
/* Comments*/  
Preprocessors and Header files  
Function header  
Global Declaration  
void main()  
{  
    Declaration part  
    Executable part  
}  
User defined functions  
{  
    Body of user defined function  
}
```

Header Files

C program depends upon some header files for function definition that are used in program. Each header file by default is extended with .h. The file should be included using #include pre-processor directive. Example is given below.

```
#include<stdio.h>
```

Global Declaration

It declares some variables that are used in more than one function. These variables are known as global variables

void main()

Every program of C language must contain main () function. The main () is starting point of every C program.

Declaration Part

It declares the variables that are used inside this particular function. The initializations of variables are also done in this section.

Executable Part

This part contains the statement blocks that may be input statement, process statement and output statement.

User defined functions

The functions defined by the user are called user defined functions. These functions are defined before or after the main() function.

Comments

It is not necessary in the program. However, to understand the flow of programs the programmer can include comments in a program.

Function header

It defines the heading or prototypes of the functions.

```

/*program to find sum of any two numbers*/
#include<stdio.h>.....Header files
#include<conio.h>
int sum(int ,int);.....Function header
int s;.....Global declaration
void main()

```

```

{
    int a,b;.....Declaration part
    printf("Enter any two numbers:");
    scanf("%d%d",&a,&b);
    s=sum(a,b);
    printf("sum=%d",s);
    getch();
}

```

Executable part

```

int sum(int x,int y)
{
    int Z;
    Z=x+y;
    return(Z);
}

```

User defined function

Program Development Tools

1. Algorithm

- An algorithm is a finite set of sequential instructions written in a natural language.
- The characteristics of algorithms are:
 - Input
 - Output
 - Unambiguous
 - Finiteness

Program Development Tools

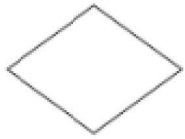
2. Flowchart

it is a diagrammatic (pictorial) representation of the instruction for solving a problem.

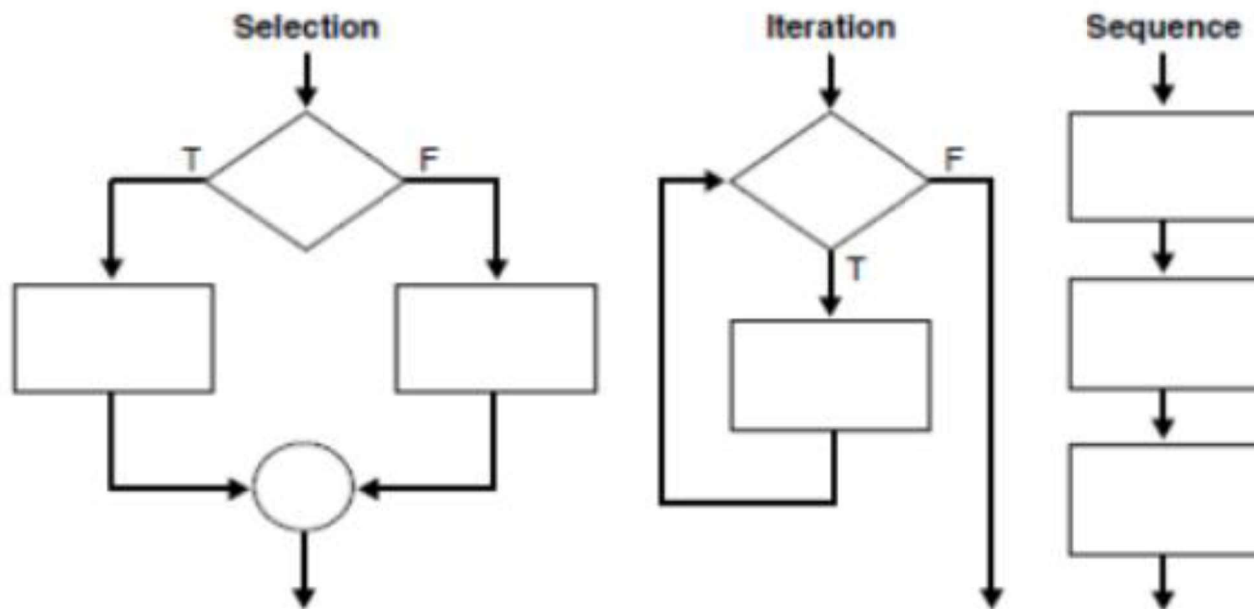
It is graphical representation of algorithm.

use various standard symbols

Symbols: Flowchart: defined by American National Standard Institute (ANSI)

Flowchart Symbol	Symbol Name	Description
	Terminal (Start or Stop)	Terminals (Oval shapes) are used to represent start and stop of the flowchart.
	Flow Lines or Arrow	Flow lines are used to connect symbols used in flowchart and indicate direction of flow.
	Input / Output	Parallelograms are used to read input data and output or display information
	Process	Rectangles are generally used to represent process. For example, Arithmetic operations, Data movement etc.
	Decision	Diamond shapes are generally used to check any condition or take decision for which there are two answers, they are, yes (true) or no (false).
	Connector	It is used connect or join flow lines.
	Annotation	It is used to provide additional information about another flowchart symbol in the form of comments or remarks.

Flowchart structure:



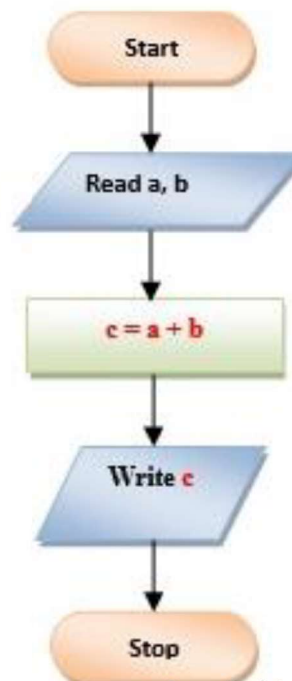
Examples

To find sum of two numbers

Algorithm

1. Start
2. Read a, b
3. $c = a + b$
4. Print or display c
5. Stop

Flowchart



Program

```
#include<stdio.h>

int main()
{
    int a, b, c;

    printf("Enter value of a: ");
    scanf("%d", &a);

    printf("Enter value of b: ");
    scanf("%d", &b);
    c = a+b;

    printf("Sum of given two numbers is: %d", c);

    return 0;
}
```

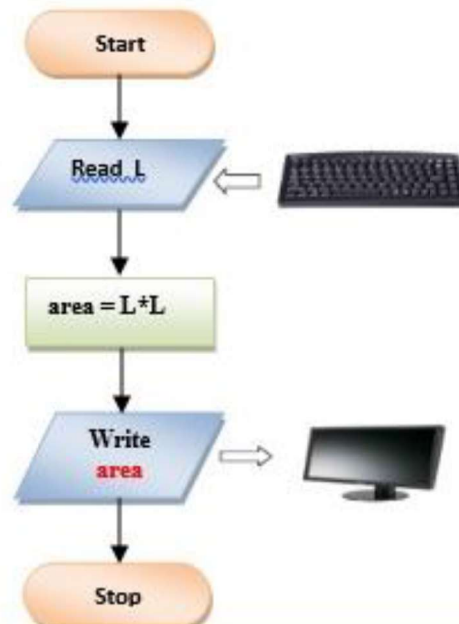
Example

Finding Area of the square

Algorithm

1. Start
2. Read length, L
3. $\text{area} = L * L$
4. Print or display **area**
5. Stop

Flowchart



Program

```
#include<stdio.h>

int main()
{
    int L, area;

    printf("Enter length of square L: ");
    scanf("%d", &L);

    area = L*L;

    printf("Area of square is: %d", area);

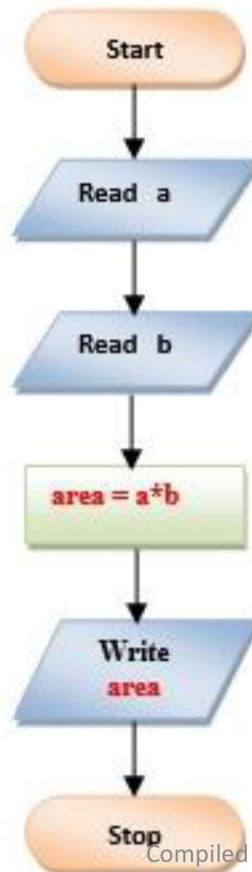
    return 0;
}
```

Finding Area of the rectangle

Algorithm

1. Start
2. Read side length, a
3. Read side length b
4. $\text{area} = a * b$
5. Print or display area
6. Stop

Flowchart



Program

```
#include<stdio.h>

int main()
{
    int a, b, area;
    printf("Enter side length a: \n");
    scanf("%d", &a);

    printf("Enter side length b: \n");
    scanf("%d", &b);

    area = a*b;

    printf("Area of rectangle is: %d ", area);

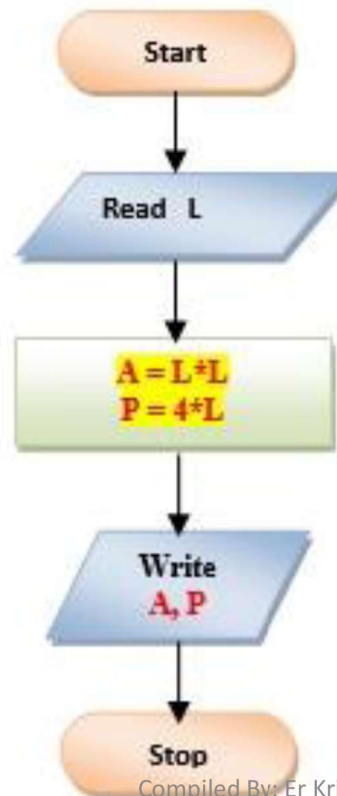
    return 0;
}
```

Find the area & perimeter of a square

Algorithm

1. Start
2. Read length L
3. Area $A = L * L$
4. Perimeter $P = 4 * L$
5. Print or display A, P
6. Stop

Flowchart



Program

```
#include<stdio.h>

int main()
{
    int L, A, P;

    printf("Enter length of a sqaure L: ");
    scanf("%d", &L);

    A = L*L;
    P = 4*L;

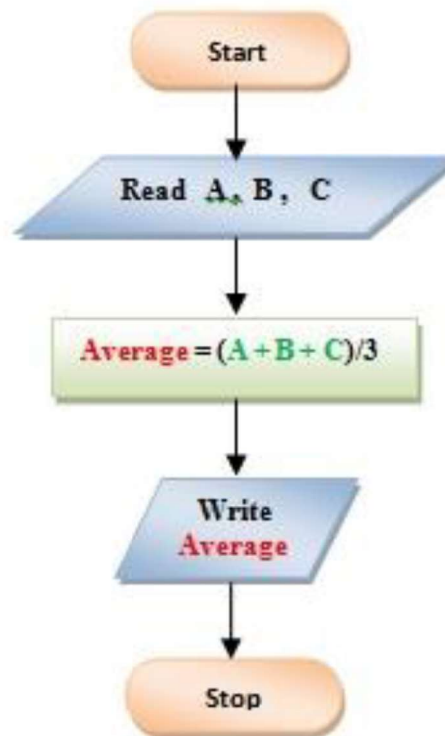
    printf("Area = %d\n", A);
    printf("Perimeter = %d", P);
    return 0;
}
```

Calculating the average for 3 numbers

Algorithm

1. Start
2. Read 3 numbers A, B, C
3. Calculate the average by the equation:
$$\text{Average} = (A + B + C)/3$$
4. Print average
5. Stop

Flowchart



Program

```
#include<stdio.h>

int main()
{
    int A, B, C;
    float Average;

    printf("Enter values of A, B, C: \n");
    scanf("%d %d %d", &A, &B, &C);

    Average = (A+B+C)/3;

    printf("Average of given 3 numbers is: %f", Average);

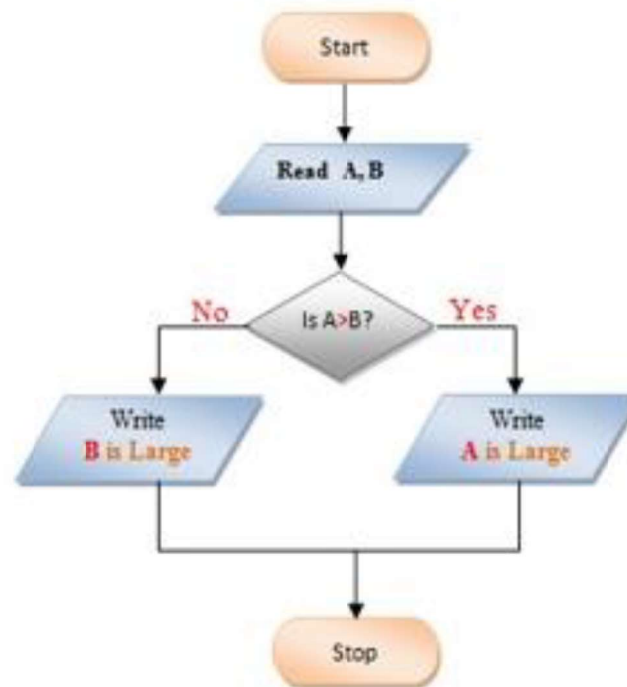
    return 0;
}
```


Greatest of two numbers

Algorithm

1. Start
2. Read A, B
3. If $A > B$ then
 Print A is large
 else
 Print B is large
4. Stop

Flowchart



Program

```
#include<stdio.h>

int main()
{
    int A, B;

    printf("Enter values of A, B: ");
    scanf("%d %d", &A, &B);

    if (A>B)
        printf("A is Larger");
    else
        printf("B is Larger");

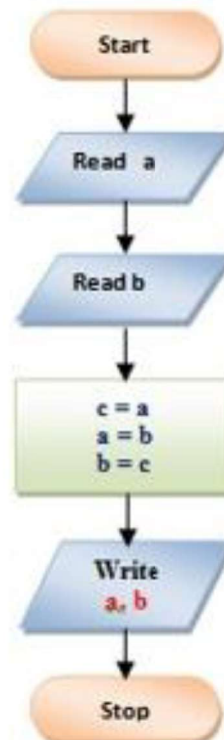
    return 0;
}
```


Interchange the value of two numbers

Algorithm

1. Start
2. Read two values into two variables a, b
3. Declare third variable, c
 $c = a$
 $a = b$
 $b = c$
4. Print or display a, b
5. Stop

Flowchart



Program

```
#include<stdio.h>

int main()
{
    int a, b, c;
    printf("Enter value of a:");
    scanf("%d", &a);

    printf("Enter value of b:");
    scanf("%d", &b);

    c = a;
    a = b;
    b = c;

    printf("Values of a & b after swapping: ");
    printf("a = %d\n", a);
    printf("b = %d", b);

    return 0;
}
```

Calculate simple interest using the expression ($SI = PNR/100$)

Algorithm

1. Start
2. Read P, N, R
3. $SI = (PNR)/100$
4. Print SI
5. Stop

Flowchart



Program

```
#include<stdio.h>

int main()
{
    int P, N, R;
    float SI;

    printf("Enter values of P, N, R: ");
    scanf("%d %d %d", &P, &N, &R);

    SI = (P*N*R/100);
    printf("Simple Interest is: %f", SI);

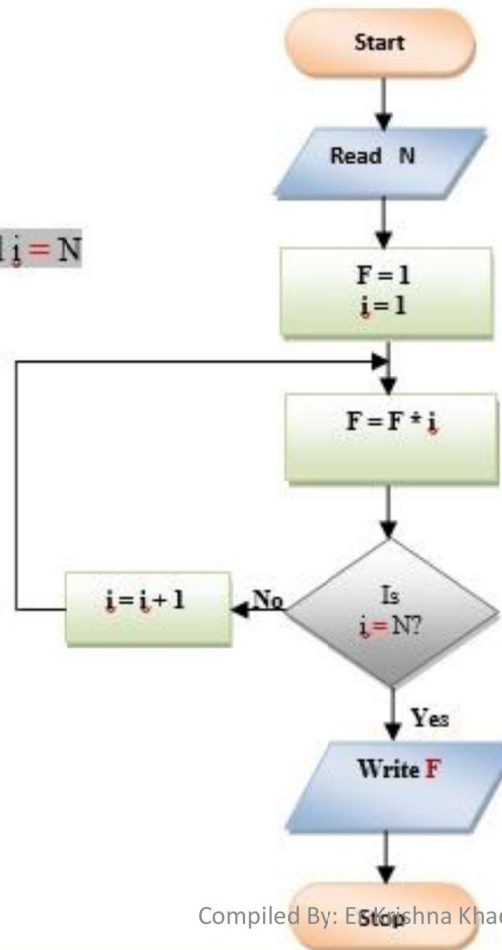
    return 0;
}
```

Draw a flowchart for computing factorial N, where $N! = 1 * 2 * 3 * \dots * N$

Algorithm

1. Start
2. Read N
3. Initialize $F = 1, i = 1$
4. $F = F * i$
5. Increment i by 1
6. Repeat step 4 & 5 until $i = N$
7. Print F
8. Stop

Flowchart



Program

```
#include<stdio.h>

int main()
{
    int N, F = 1, i;

    printf("Enter value of N: ");
    scanf("%d", &N);

    for(i=1; i<=N;i++)
    {
        F = F * i;
    }

    printf("Factorial of N is: %d", F);

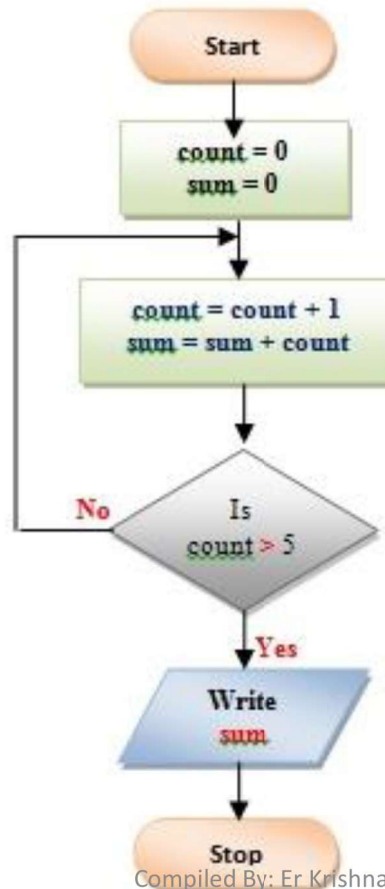
    return 0;
}
```

Find the Sum of First Five Natural Numbers

Algorithm

1. Start
2. Initialize $\text{count} = 0$, $\text{sum} = 0$
3. $\text{count} = \text{count} + 1$
4. $\text{sum} = \text{sum} + \text{count}$
5. Repeat steps 3,4 until $\text{count} > 5$
6. Print sum
7. Stop

Flowchart



Program

```
#include<stdio.h>

int main()
{
    int count, sum;
    sum = 0;

    for (count = 1; count<=5; count++)
    {
        sum = sum +count;
    }

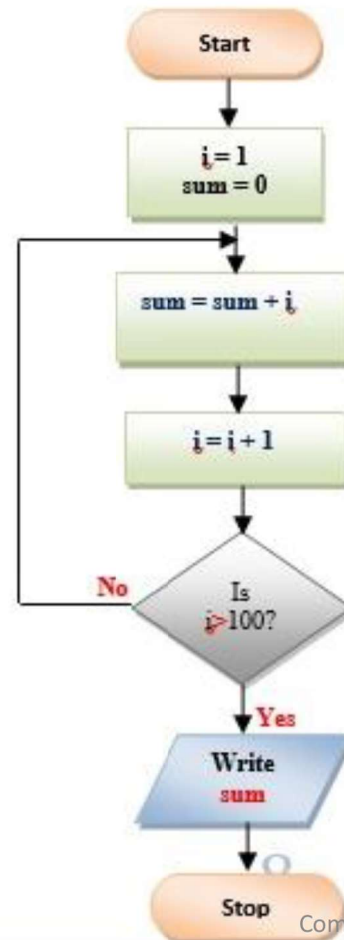
    printf("Sum of 1st 5 numbers is: %d", sum);
    return 0;
}
```

Calculating sum of integers 1 to 100

Algorithm

1. Start
2. Initialize count $i = 1$, $sum = 0$
3. $sum = sum + i$
4. Increment i by 1
5. Repeat steps 3 & 4 until $i > 100$
6. Print sum
7. Stop

Flowchart



Program

```
#include<stdio.h>

int main()
{
    int i, sum;

    sum = 0;
    for(i=1; i<101; i++)
    {
        sum = sum + i;
    }

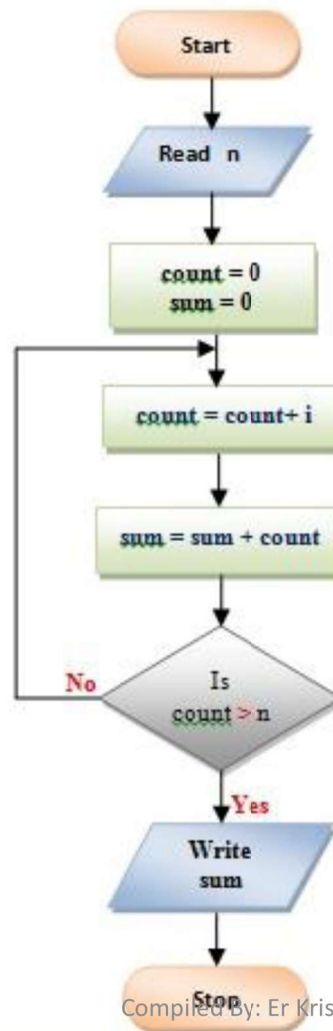
    printf("Sum of integers from 1 to 100 is: %d", sum);
    return 0;
}
```

To find the sum of **n** natural Numbers

Algorithm

1. Start
2. Read n
3. count=0
4. sum=0
5. count = count + 1
6. sum = sum + count
7. Repeat steps 5 & 6 until count > n
8. Print sum
9. Stop

Flowchart



Program

```
#include<stdio.h>

int main()
{
    int n, i, sum;

    sum = 0;

    printf("Enter value of n: ");
    scanf("%d", &n);

    for(i=1; i<=n; i++)
    {
        sum = sum + i;
    }

    printf("Sum of n natural numbers is: %d", sum);

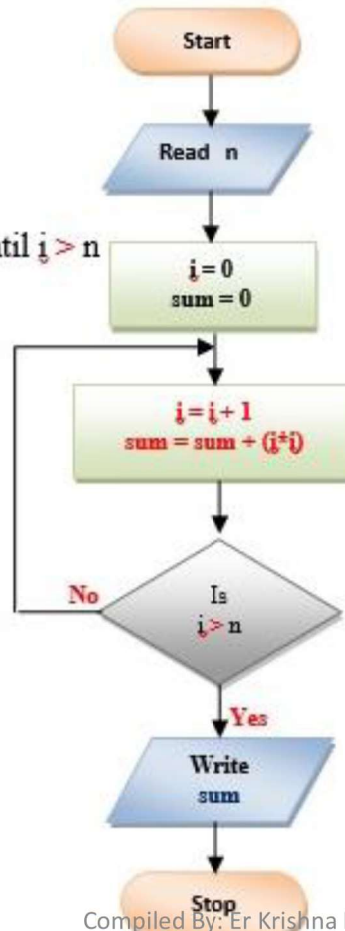
    return 0;
}
```


Sum of squares of n natural numbers

Algorithm

1. Start
2. Read n
3. $i = 0$, $sum = 0$
4. $i = i + 1$
5. $sum = sum + (i*i)$
6. Repeat steps 4 and 5 until $i > n$
7. Print sum
8. Stop

Flowchart



Program

```
#include<stdio.h>

int main()
{
    int n, i, sum;

    printf("Enter value of n: ");
    scanf("%d", &n);

    sum = 0;
    for(i=1; i<=n; i++)
    {
        sum = sum + i*i;
    }

    printf("Sum of square of integers ");
    printf("up to n is: %d", sum);

    return 0;
}
```

H/W:

- Write an algorithm and draw a flowchart to input a number and check if it is positive , negative or zero.
- Write an algorithm and draw a flowchart to find the largest among three numbers.
- Write an algorithm and draw a flowchart to display the number from 1 to 5.